

tsw-steam-bridge

User Manual

September 18, 2026

A Zig bridge between Steam Input and Train Sim World: it reads real Steam Input digital/analog actions and drives Train Sim World's cab controls directly through its external HTTP API, instead of going through Steam Input's keyboard/mouse emulation.

Why this exists: keyboard emulation flattens two things Train Sim World needs kept apart. Throttle and brake want continuous values, not just “tapped” or “held”. And safety controls like AWS (Automatic Warning System) reset and DSD (Driver's Safety Device / vigilance) reset are separate physical controls in the real cab that often end up sharing or fighting over the same key in a simple keybind scheme. This bridge keeps them independent by talking to Steam Input's real action API and to Train Sim World's real control API, with nothing synthetic in between.

Contents

1	Overview	3
2	Prerequisites	3
3	Enabling Train Sim World's external API	3
4	Vendoring the Steamworks SDK	4
5	Building	4
6	tsw-cli — exploring and testing the API	5
6.1	Checking the API is reachable	5
6.2	Listing controls	5
6.3	Reading and writing values	5
6.4	Building a locomotive profile	5
7	steam-bridge — running the live bridge	5
8	tsw-gui — the live dashboard	6
9	Troubleshooting	7
10	Reference: profile file format	7

1 Overview

The project has three executables, built from the same source tree:

Executable	Purpose
<code>tsw-cli</code>	Pure Zig, no external dependencies. Talks to Train Sim World’s <code>-HTTPAPI</code> over plain HTTP. Use it to check the API is reachable, explore a locomotive’s control tree, and hand-test individual values.
<code>steam-bridge</code>	The actual bridge process. Links against the Steamworks SDK to read real Steam Input actions and writes their values into Train Sim World via the same client code <code>tsw-cli</code> uses.
<code>tsw-gui</code>	A live dashboard shell built on Dear ImGui, themed from your Omarchy colors and font, with dockable panels.

Control names are **not consistent across locomotives** — not even across versions of the same class — so there is no single, universal “AWS reset” path in Train Sim World’s API. Instead, each locomotive gets its own small JSON *profile* under `profiles/`, mapping a stable set of logical control names (`throttle`, `aws_reset`, `dsd_reset`, ...) onto that particular locomotive’s real node paths. You build these up over time, one locomotive at a time, using `tsw-cli discover`.

2 Prerequisites

- **Zig 0.16.0 or newer.**
- **Train Sim World**, launched with the `-HTTPAPI` Steam launch option (see Section 3).
- **The Steamworks SDK**, vendored locally (see Section 4) — needed to build `steam-bridge`.
- **A Steam client**, running and logged in, for `steam-bridge` and any Steam Input functionality.

`tsw-cli` needs none of the above except Zig itself, so it is the easiest place to start.

3 Enabling Train Sim World’s external API

Train Sim World ships an external control API that is not officially documented by Dovetail Games, but is well understood from community reverse-engineering. To enable it:

1. In Steam, open Train Sim World’s properties and add `-HTTPAPI` to the launch options.
2. Launch the game once. This writes a file called `CommAPIKey.txt` under `Documents/MyGames/TrainSimWorld<N>/Saved/Config/` (where `<N>` is the game’s version number).
3. If you’re running Train Sim World through Proton, that path is inside the Proton prefix, typically under something like
`~/.local/share/Steam/steamapps/compatdata/<appid>/pfx/
drive_c/users/steamuser/MyDocuments/MyGames/....`
The quickest way to find it is:

```
find ~/.steam ~/.local/share/Steam -iname CommAPIKey.txt
```

Every request to the API must include the contents of that file in a `DTGCommKey` HTTP header — the tools in this project handle that for you once you point them at the file with `--key-file`.

The API was observed listening on `http://localhost:31270` on Train Sim World 5. This is not officially documented and may differ on later versions; run `tsw-cli info` to confirm (Section 6).

4 Vendoring the Steamworks SDK

`steam-bridge` needs Valve’s Steamworks SDK, which is not redistributable and must be downloaded yourself:

1. Sign in with any Steam account at <https://partner.steamgames.com/downloads/list> and download `steamworks_sdk.zip`.
2. Extract it into the project so that `vendor/sdk/public/...` and `vendor/sdk/redistributable_bin/...` exist.

`vendor/sdk/` is git-ignored — it is a large, license-encumbered redistributable, not part of the project’s source. `zig build` will still build `tsw-cli` and `tsw-gui` without it; it only skips `steam-bridge`.

Why app id 480

Steam Input’s API needs to run as some recognized Steam application — `SteamAPI_Init` needs a `steam_appid.txt` file (or a `SteamAppId` environment variable) naming one. For a personal, unpublished tool like this, the standard approach used across the community is Valve’s public “Spacewar” test application, id 480. `steam-bridge` writes `steam_appid.txt` containing 480 automatically on first run if it is missing. If you ever publish this project under your own Steam app id, replace that file’s contents and regenerate `manifest/game_actions_480.vdf` for your id.

5 Building

From the project root:

```
zig build
```

This builds `tsw-cli` and `tsw-gui` unconditionally, and `steam-bridge` if the Steamworks SDK has been vendored (Section 4). The GUI’s own dependencies (`zgui`, `zglfw`, `zopengl`) are fetched automatically the first time you build.

Binaries land in `zig-out/bin/`. You can also run them directly through the build system, which forwards arguments after `--`:

```
zig build run-cli -- info --key-file /path/to/CommAPIKey.txt
zig build run-bridge -- --key-file /path/to/CommAPIKey.txt
zig build run-gui
```

A note if you’re building this yourself on a similar system

On some current Linux systems, Zig 0.16.0’s own linker cannot yet handle the `.sframe` unwind sections that current `glibc/gcc` toolchains emit — any `libc`-linked Zig binary fails to link with an error mentioning `unhandled relocation type R_X86_64_PC64 ... .sframe`. If you hit this, it is a toolchain issue, not a problem with your setup specifically; this project’s own `build.zig` already works around it for `steam-bridge` and `tsw-gui` by linking those two through the system `cc/c++` instead of Zig’s built-in linker.

6 tsw-cli — exploring and testing the API

tsw-cli is the tool you reach for first: it talks to Train Sim World directly over HTTP and does nothing with Steam Input at all.

6.1 Checking the API is reachable

```
tsw-cli --key-file /path/to/CommAPIKey.txt info
```

A successful response confirms the port and prints the game's name and build number, which is the fastest way to sanity-check that `-HTTPAPI` took effect and that the port hasn't moved on your version of the game.

6.2 Listing controls

```
tsw-cli --key-file <key> list CurrentDrivableActor
```

Lists the child nodes and endpoints of whatever locomotive you are currently driving — this is how you discover what is actually controllable on a given loco, since names vary by locomotive class.

6.3 Reading and writing values

```
tsw-cli --key-file <key> get CurrentDrivableActor/Throttle_F.Value  
tsw-cli --key-file <key> set CurrentDrivableActor/Pantograph_F.Value true
```

Use `set` carefully and only while stationary until you know what a control actually does — some are simple booleans, others expect a float in a particular range.

6.4 Building a locomotive profile

```
tsw-cli --key-file <key> discover
```

Walks the current locomotive's writable controls and writes a skeleton profile to `profiles/<ObjectClass>.json`, where `<ObjectClass>` is that locomotive's internal class name. The skeleton lists every candidate writable control under its raw node name; you then:

1. Test each candidate path with `tsw-cli set` while parked, and observe what happens in the cab.
2. Rename the JSON key to the matching logical name the bridge expects. The full, current list — covering core cab controls plus British (AWS/DSD/TPWS/DRA), German (SIFA/PZB/LZB) and ETCS actions — is kept in `profiles/README.md` rather than duplicated here, since it grows as the bridge's control vocabulary does.
3. Fix "kind" to "float" for anything that isn't a plain true/false.
4. Delete entries you don't care about mapping — an incomplete profile is fine; the bridge simply skips logical controls that aren't mapped for the current locomotive.

7 steam-bridge — running the live bridge

Once you have a locomotive profile and the Steamworks SDK vendored:

```
zig build run-bridge -- --key-file /path/to/CommAPIKey.txt
```

On startup, **steam-bridge** initializes the Steamworks API, activates its own Driving Steam Input action set (defined in `manifest/game_actions_480.vdf`, entirely separate from Train Sim World’s own Steam Input configuration), and begins polling. Every tick it:

1. Reads the current locomotive’s class from Train Sim World and loads the matching profile from `profiles/`.
2. Polls each logical Steam Input action — `throttle`, `brake` and `combined_power_brake` as analog axes or stepped detents depending on the active profile’s setting for that lever, the rest as digital buttons.
3. For anything that changed and has a mapping in the active profile, sends the new value to Train Sim World via `PATCH /set/...`

Notched vs. notchless levers

Real locomotives have both kinds of throttle/brake handle, and holding an auto-centering analog stick at one precise deflection to maintain line speed for an hour is uncomfortable at best. So each of `throttle`, `brake` and `combined_power_brake` can instead be driven by a pair of up/down detent actions (`throttle_up/throttle_down`, etc.) that **steam-bridge** accumulates into an absolute position itself — snapped to a fixed number of notches, or stepped by a tunable amount for a notchless lever. This is controlled per-lever by that control’s “mode” field in its locomotive profile (“absolute”, “notched” with a “notches” count, or “notchless” with a “step.size”); see `profiles/README.md` for the full field reference. The default step size for notchless levers can be overridden globally with `steam-bridge --step-size <value>`.

A detent-based binding pairs naturally with Steam Input’s trackpad “Scroll Wheel” mode set to circular swipe, which turns a flat trackpad into a haptic rotary encoder — but any pair of buttons or paddles works just as well.

Current status: Steam’s own API confirms successful initialization against a real, running Steam client, but activating the custom action manifest is not yet reliably working end-to-end — this is the active area of development. See the project README’s “Known gaps” section for the latest status.

8 tsw-gui — the live dashboard

tsw-gui is a windowed dashboard with dockable panels, built on Dear ImGui. Launch it with:

```
./scripts/run-gui.sh
```

This wrapper resolves whatever monospace font Omarchy currently has set (`omarchy font current`, resolved to a real font file via `fc-match`) and passes it through, so the dashboard’s font always matches your system font without the program itself needing to know anything about Omarchy or fontconfig. Colors are read directly from your active Omarchy theme’s `colors.toml` at startup, so the panel follows whichever theme is active system-wide — nothing is hardcoded.

Figure 1 shows the dashboard’s “Controller” panel, themed from the Coaster theme’s palette and set in the SlabMono Nerd Font variant that was active on this machine at the time.

Panels can be dragged by their title bar to rearrange or dock them together. Dear ImGui saves this layout automatically to an `imgui.ini` file in the working directory and restores it the next time you launch **tsw-gui**.

Current status: this is a windowing/theming/layout scaffold. The panels shown are representative of the intended layout but are not yet wired to a running **steam-bridge**’s live data —

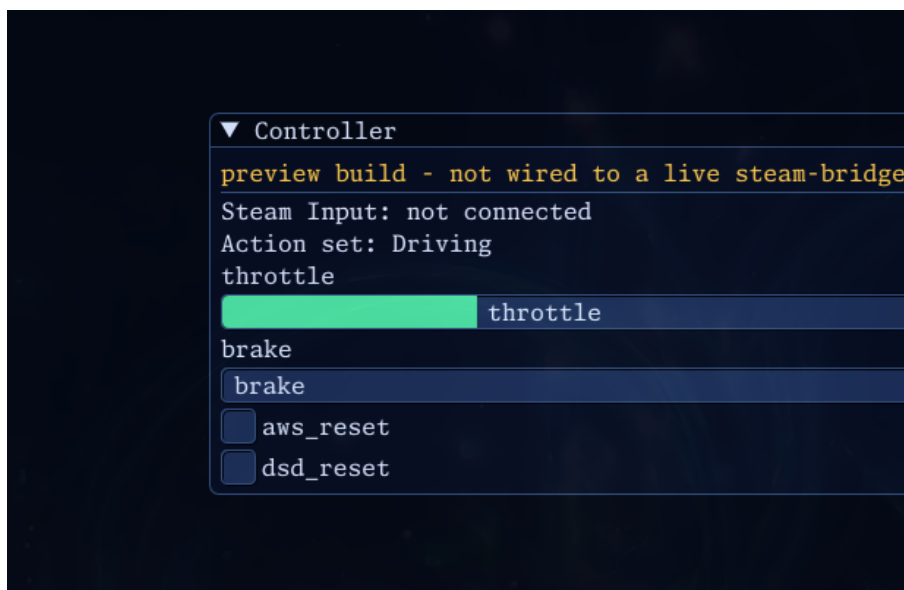


Figure 1: The tsw-gui Controller panel, docked and themed from the active Omarchy theme.

that is the natural next step once Steam Input manifest activation is working end to end.

9 Troubleshooting

Symptom	Likely cause / fix
<code>tsw-cli info</code> fails to connect	Check <code>-HTTPAPI</code> is actually in Train Sim World's launch options and the game has been launched at least once since adding it. Confirm the key file path. Try the base URL override if your game version uses a different port.
Steam Input fails with <code>NoSteamClient</code>	Steam isn't running, or isn't fully started yet. Make sure Steam is open and you are logged in before starting <code>steam-bridge</code> .
<code>zig build</code> skips <code>steam-bridge</code>	The Steamworks SDK hasn't been vendored under <code>vendor/sdk/</code> — see Section 4.
A control does nothing when set	Not every node listed by <code>discover</code> is meaningful — some are read-only telemetry mirrors or apply only to specific train sub-systems. Cross-check against what changes in the cab.
<code>tsw-gui</code> shows the built-in font instead of my font	Launch it via <code>scripts/run-gui.sh</code> , or pass <code>--font-file</code> yourself with a real <code>.ttf/.otf</code> path.

10 Reference: profile file format

Profiles live at `profiles/<ObjectClass>.json`:

```
{
  "objectClass": "RVM_CRG_DB_BR101_C",
```

```
"displayName": "DB BR 101",
"controls": {
  "throttle": { "path": "CurrentDrivableActor/Throttle_F.Value", "kind": "float" },
  "aws_reset": { "path": "CurrentDrivableActor/AWS_F.Value", "kind": "bool" }
}
```

kind is either "bool" or "float" and controls how the bridge formats the value it sends to PATCH /set/.... A logical control with no entry is simply never sent for that locomotive.